Absolutely! Let's dive into **Python modules from basic to advanced**, step by step, with examples, exercises, and real-world insights. I'll organize it in **levels** so you can progress from beginner to advanced.

Python Modules – Complete Guide

1️⃣ Basic Level: Understanding Modules

Definition

A **module** is a Python file (.py) containing **functions, variables, or classes** that you can reuse in other Python programs.

Creating a Simple Module

Create a file mymodule.py:

```
# mymodule.py


def greet(name):
    return f"Hello, {name}!"


pi = 3.1416
```

Use it in main.py:

```
# main.py


import mymodule


print(mymodule.greet("Ali"))
print(mymodule.pi)
```

Key Concepts:

- import module_name → Import the whole module

- from module_name import function_name → Import specific functions
- from module_name import * → Import everything (not recommended for big projects)

Exercise: Basic

1. Create a module calculator.py with functions: add, subtract, multiply, divide.
2. Test it in another file by importing it.

2️⃣ Intermediate Level: Working with Built-in Modules

Python comes with **hundreds of built-in modules**. Some important ones:

Module	Use Case
math	Mathematical operations (sqrt, factorial, pi)
random	Random numbers, choices, shuffling
os	Interacting with files & folders
sys	Command-line arguments, system info
datetime	Date & time manipulation
time	Working with timestamps & delays
json	Handling JSON data
re	Regular expressions

Examples

Math Module

```
import math


print(math.sqrt(25))     # 5.0
print(math.factorial(6))  # 720
```

Random Module

```
import random


print(random.randint(1, 100))        # Random integer
print(random.choice(['a','b','c']))   # Random element
```

OS Module

```
import os


print(os.getcwd())  # Current working directory
os.mkdir("test_folder")  # Create a folder
```

Exercise: Intermediate

1. Use datetime to print today's date and current time.
2. Use random to simulate rolling a dice 20 times and print results.
3. Use os to list all files in a folder.

3️⃣ Advanced Level: Custom Modules, Packages, and Namespaces

3.1 Creating Advanced Modules

A module can contain:

- **Functions**
- **Variables**
- **Classes**
- **Imports from other modules**

Example advanced_module.py:

```
# advanced_module.py
```

```
class Calculator:
    def add(self, a, b):
        return a + b
    def subtract(self, a, b):
        return a - b

def greet(name):
    return f"Hello, {name}!"

PI = 3.1416
```

Use in main.py:

```
from advanced_module import Calculator, greet, PI

calc = Calculator()
print(calc.add(10, 5))
print(greet("Ali"))
print(PI)
```

3.2 Packages (Modules inside Folders)

A **package** is a folder with multiple modules and an optional __init__.py.

Example structure:

```
mypackage/
    __init__.py
    math_utils.py
    string_utils.py
```

math_utils.py

```
def add(a, b):
    return a + b

def multiply(a, b):
    return a * b
```

string_utils.py

```
def capitalize(text):
    return text.upper()
```

main.py

```
from mypackage import math_utils, string_utils

print(math_utils.add(5, 6))       # 11
print(string_utils.capitalize("hello"))  # HELLO
```

3.3 Module Aliases

You can give a module a short alias using as:

```
import math as m
print(m.sqrt(49))  # 7.0
```

3.4 Reloading Modules (During Development)

If you change a module while your program is running:

```
import importlib
import mymodule

importlib.reload(mymodule)
```

3.5 Private Functions and Variables in Modules

Prefix with _ to indicate **private**, meaning intended for internal use:

```
# mymodule.py

def _hidden_function():

    return "Secret!"


def public_function():

    return "Visible"
```

Only public_function should be used outside the module.

3.6 Using __name__ == "__main__" in Modules

Allows a module to **run as a script or be imported** safely:

```
# mymodule.py


def greet(name):

    return f"Hello, {name}"


if __name__ == "__main__":

    print(greet("Ali"))
```

- When imported, the code inside if __name__ == "__main__": **does not run**.
- When run directly, it **executes the code**.

Exercise: Advanced

1. Create a package mypackage with modules:
    - math_ops.py → add, subtract, multiply
    - string_ops.py → reverse string, capitalize

2. Create main.py to use both modules.
3. Use _hidden_function in one module and show it's not accessible outside.
4. Add __name__ == "__main__" in one module and test behavior when importing vs running.

4️⃣ Real-World Applications of Modules

Scenario	Modules / Packages Used
Data Analysis	numpy, pandas, matplotlib, seaborn
Web Scraping	requests, beautifulsoup4, lxml
Web Development	flask, django
Machine Learning	tensorflow, torch, scikit-learn
Automation Scripts	os, sys, shutil
Game Development	pygame
APIs	requests, json

If you want, I can now **create a full diagram showing Functions → Modules → Packages → Programs → Projects → Real-world Applications**, which will **link everything visually** from basic to advanced.

Do you want me to do that?